# JavaScript Compiler Architecture

This document explains JavaScript compiler architecture with diagrams and examples.

## 1. What is a Compiler?

A compiler converts human-readable code into computer-understandable instructions.

### First Important Thing About JavaScript

JavaScript is not a fully compiled language like C or C++.

**JavaScript uses:**

- **Parser**
- **Compiler**
- **Interpreter**
- **JIT (Just-In-Time) Compilation**

**Modern JavaScript engines like:**

- V8
- SpiderMonkey

# Real JavaScript Flow

## 2. Tokenizer (Lexical Analysis)

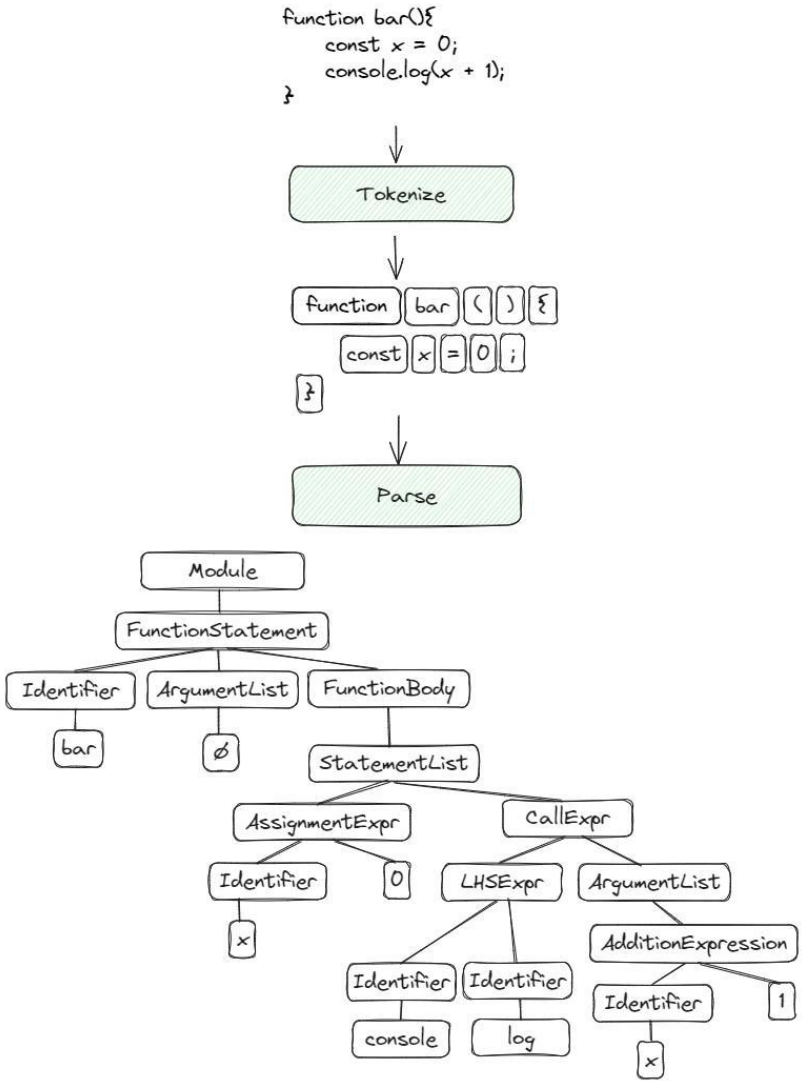
Tokenizer breaks JavaScript code into small parts called tokens.

Example:
let x = 10 + 20;

Tokens:
let | x | = | 10 | + | 20 | ;

**Tokens Output**

KEYWORD → let
IDENTIFIER → total
OPERATOR → =
NUMBER → 10
OPERATOR → +
NUMBER → 20
SYMBOL → ;




## 3. Parser

Parser checks whether syntax is correct or not.

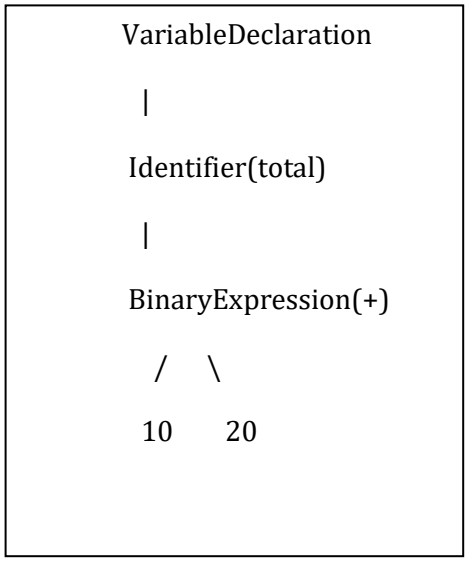
parser checks:

- Is syntax valid?
- Is grammar correct?

Correct: let age = 20;  Wrong: let = age 20;

**Parser's Main Job**

Parser converts tokens into: **AST (Abstract Syntax Tree)**

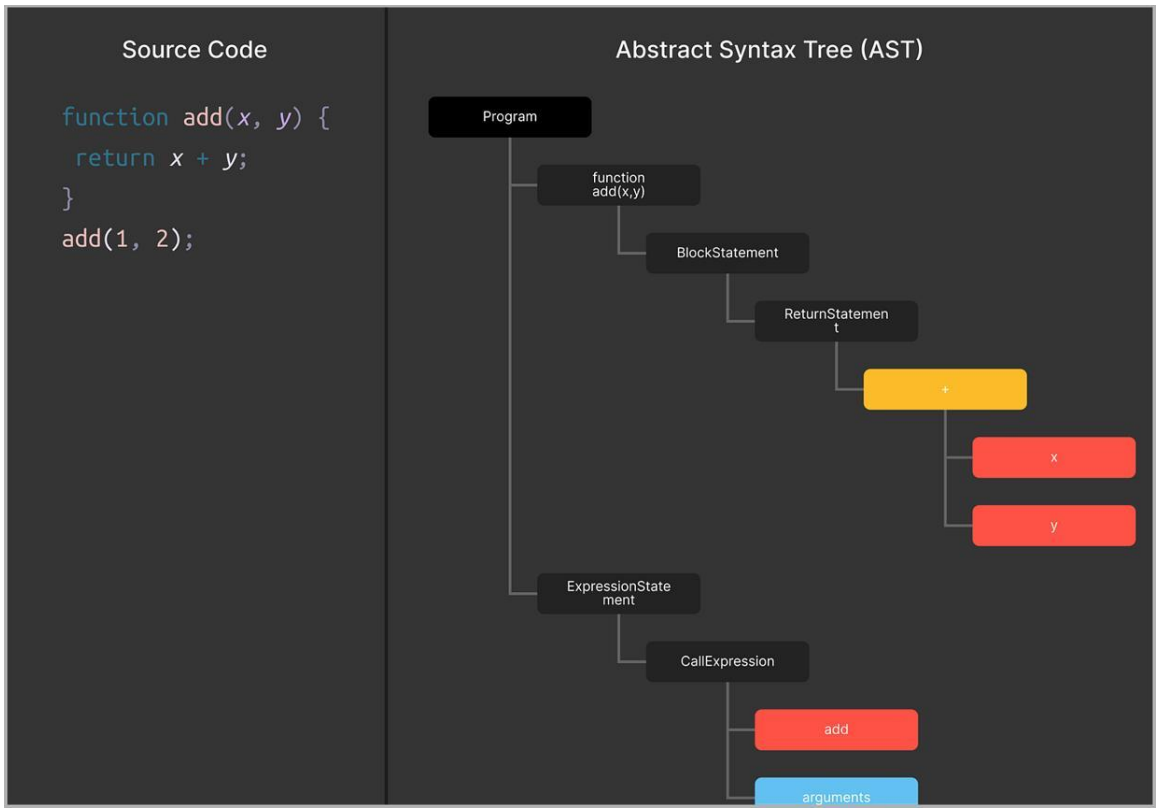


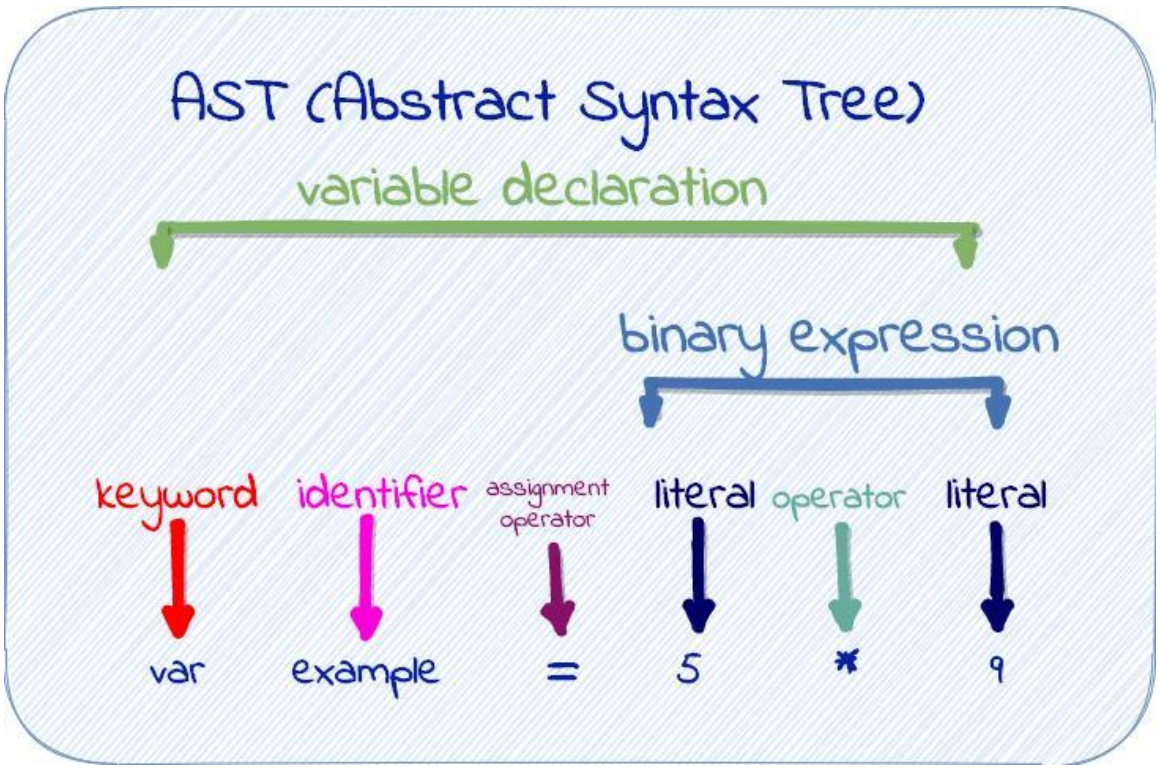
## 4. AST (Abstract Syntax Tree)

AST is a tree structure representation of code.

JavaScript engine understands code using AST.

let total = 10 + 20;






## Why AST is Important

AST helps engine understand:

- operations
- variable declarations
- loops
- conditions
- functions

Everything becomes structured.

Without AST:

Computer cannot understand code relationships.

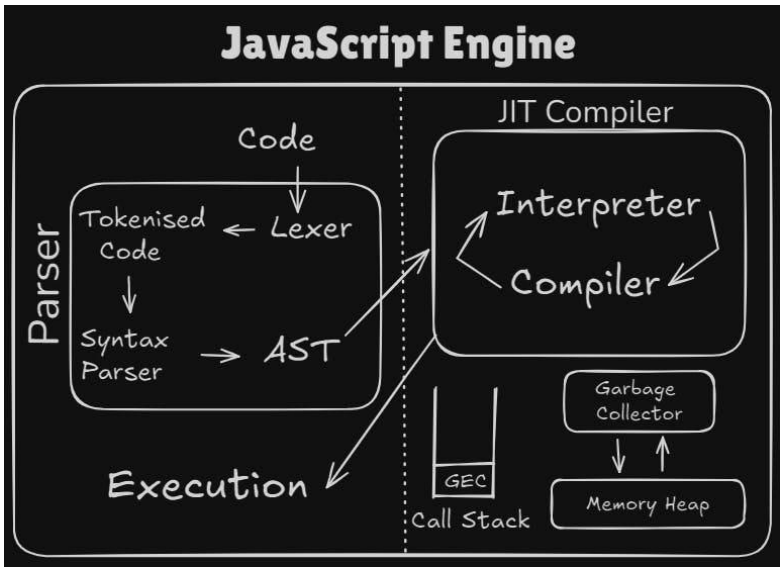
## 5. Bytecode

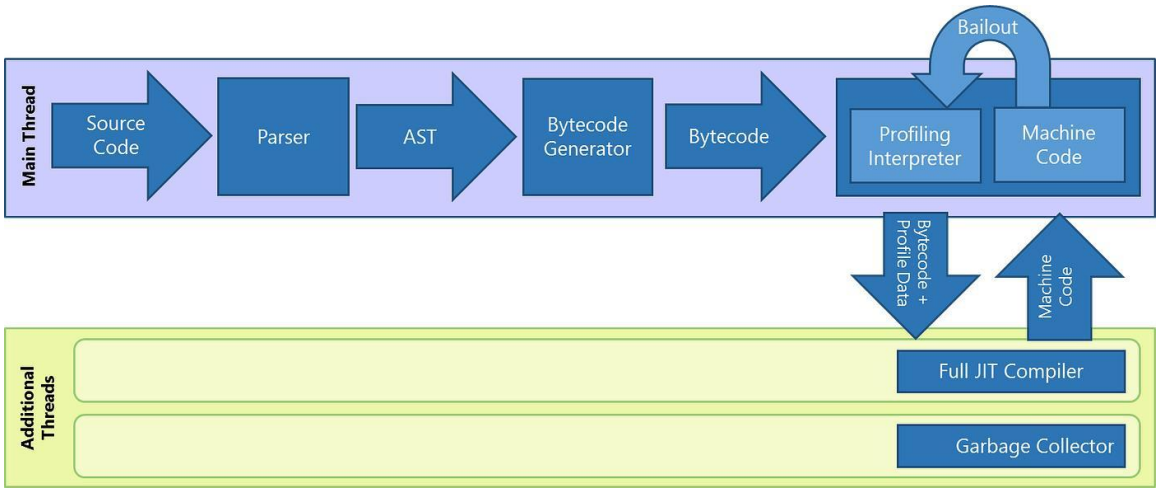
Bytecode is intermediate low-level instructions generated by the JavaScript engine.

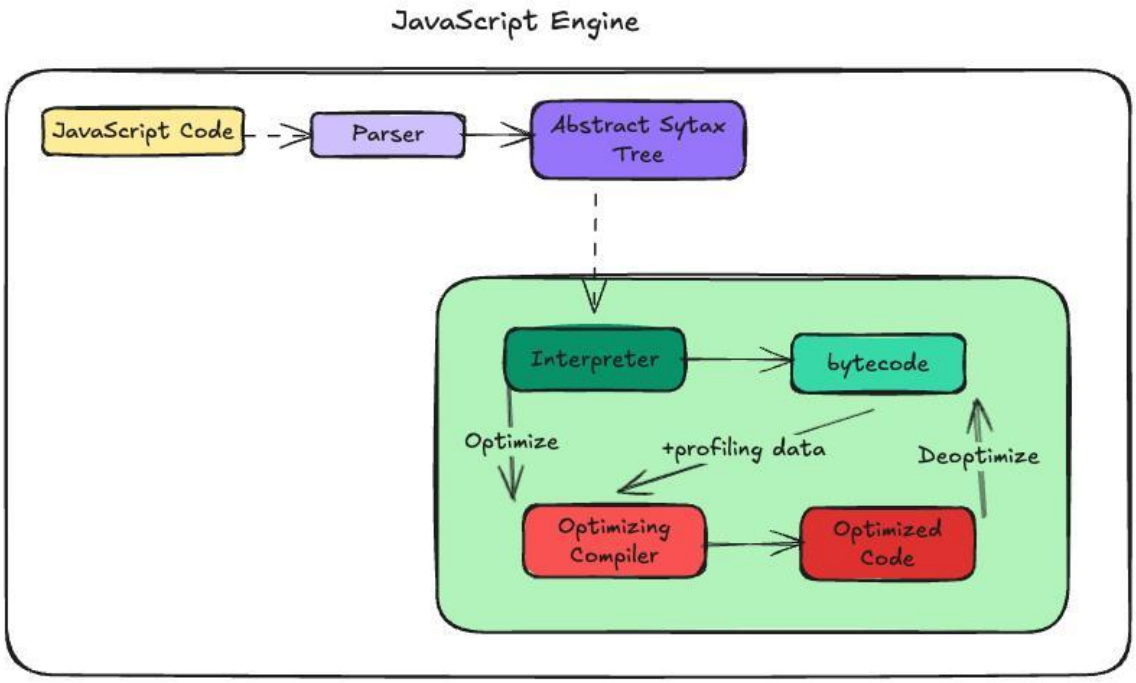
Modern JS engines convert AST into:

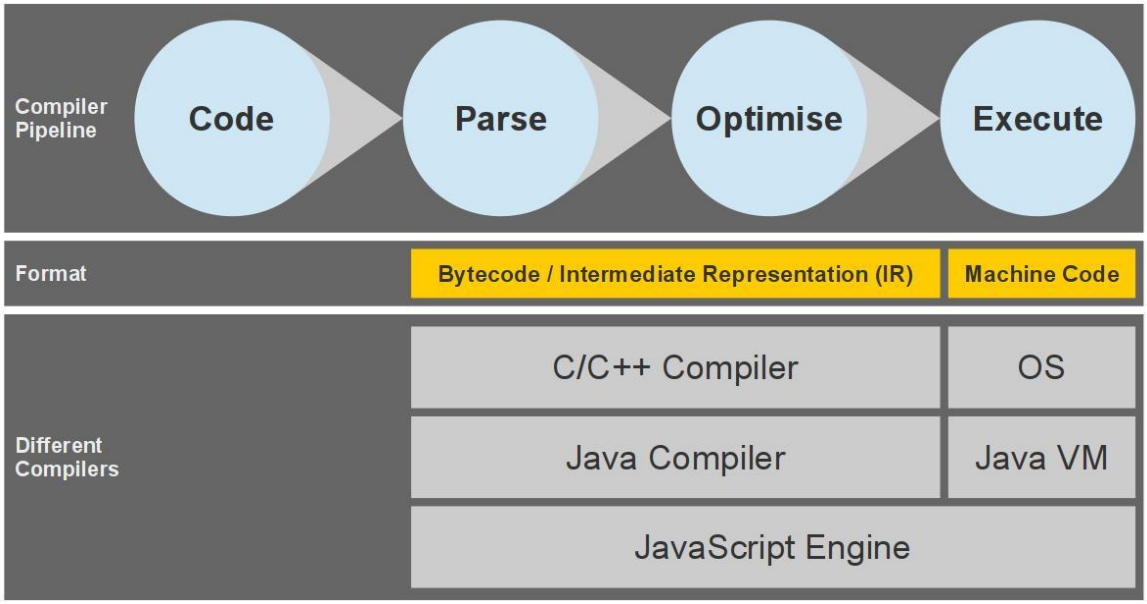
**Bytecode is:**

- lower-level instructions
- easier for machine to understand
- faster than raw JS




Main Thread
Source Code
Parser
AST
Bytecode Generator
Bytecode
Bailout
Profiling Interpreter
Machine Code
Bytecode + Profile Data
Machine Code
Additional Threads
Full JIT Compiler
Garbage Collector


JavaScript Engine
JavaScript Code
Parser
Abstract Sytax Tree
Interpreter
bytecode
Optimize
+profiling data
Deoptimize
Optimizing Compiler
Optimized Code




## 6. Interpreter

Interpreter executes code line-by-line using bytecode.

| **Compiler** | **Interpreter** |
|---|---|
| **Converts whole code first** | Runs line by line |
| **Faster execution** | Slower execution |
| **Example: C++** | Example: Old JavaScript |
| **Creates executable file** | No executable file |

**"JavaScript uses both interpretation and JIT compilation."**

## 7. JIT Compiler

Modern JavaScript engines use:

**JIT = Just-In-Time Compiler**

This is the smart optimization system.

JIT Compiler optimizes frequently used code into fast machine code.

## 8. Machine Code

Converted into CPU instructions.

CPU executes instantly.

### JavaScript Architecture Flow


EX:

```
for(let i=0; i<1000000; i++) {
    total += i;
}
```

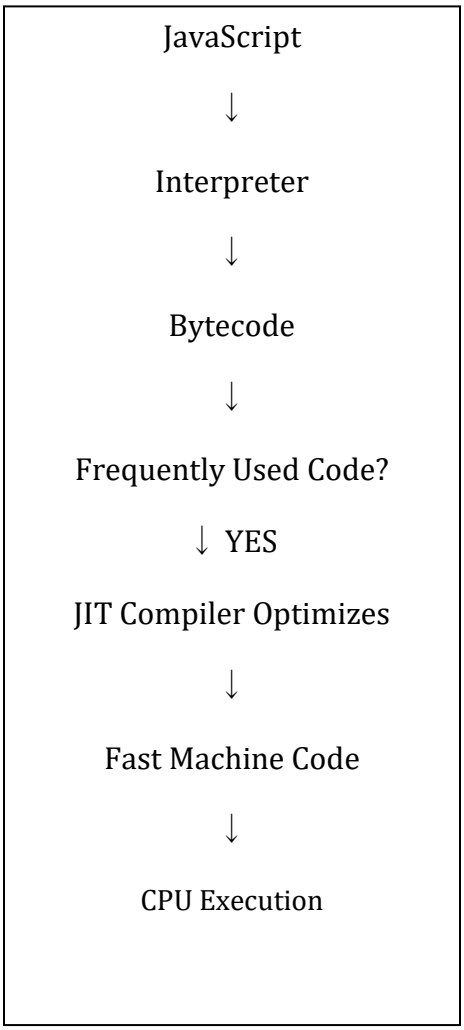
Engine notices:

"This loop runs many times."

So JIT compiles it into highly optimized machine code.

This makes execution much faster.

## JIT Flow


JavaScript
↓
Interpreter
↓
Bytecode
↓
Frequently Used Code?
↓ YES
JIT Compiler Optimizes
↓
Fast Machine Code
↓
CPU Execution

**Inside Google Chrome**

Chrome uses:

V8

V8 components:

| **Component** | **Work** |
|---|---|
| **Scanner** | Creates tokens |
| **Parser** | Creates AST |
| **Ignition** | Generates bytecode |
| **TurboFan** | Optimizes machine code |

## Most Important Understanding

| **Term** | **Meaning** |
|---|---|
| Tokenizer | Breaks code into tokens |
| Parser | Checks syntax |
| AST | Tree representation of code |
| Interpreter | Executes instructions |
| Bytecode | Intermediate low-level code |
| JIT Compiler | Optimizes repeated code |
| Machine Code | Final CPU instructions |

## Final Super Simple Analogy

Imagine restaurant workflow:

| JavaScript Engine | Restaurant |
|---|---|
| **Source Code** | **Customer order** |
| **Tokenizer** | **Splitting words** |
| **Parser** | **Checking order correctness** |
| **AST** | **Structured recipe** |
| **Bytecode** | **Kitchen instructions** |
| **Interpreter** | **Chef cooking** |
| **JIT Compiler** | **Faster expert chef** |
| **Machine Code** | **Final food served** |